

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 1: Introduction to Databases

Author: Nikhil Singh (Learning Notes)

Mentor: ChatGPT

1.1 Why Databases Exist

Imagine building a social media application. Users sign up, send messages, upload photos and interact with each other. Storing this information only in JavaScript arrays works only until the server restarts. Arrays are stored in RAM and are temporary.

A database stores information permanently, allowing data to survive server restarts and enabling millions of users to access the same data reliably.

Example JavaScript array:

```
const users = [  
  { id: 1, name: "Nikhil" },  
  { id: 2, name: "Rahul" }  
];
```

Real-World Examples

Instagram stores users, followers, posts, comments, likes and messages.

Amazon stores products, customers, orders, addresses and payments.

Banks store customers, accounts, transactions and loans.

Definition

A database is an organized collection of data that allows us to store, retrieve, update and delete information efficiently.

These four operations are called CRUD:

- Create
- Read
- Update
- Delete

1.2 What is a DBMS?

A Database Management System (DBMS) is software that manages databases and provides a safe way for applications to store and retrieve data.

Examples:

- PostgreSQL
- MySQL
- SQLite
- Oracle
- Microsoft SQL Server

Applications communicate with the DBMS, not directly with the database files.

1.3 What is an RDBMS?

PostgreSQL is a Relational Database Management System (RDBMS). Data is stored in tables that are related to one another.

Example:

Users

id	name
1	Nikhil
2	Rahul

Orders

id	product	user_id
1	Laptop	1
2	Mouse	1
3	Keyboard	2

users.id → orders.user_id creates the relationship.

1.4 What is SQL?

SQL stands for Structured Query Language. It is the language used to communicate with relational databases.

Examples:

```
SELECT * FROM users;
```

```
INSERT INTO users(name)  
VALUES ('Nikhil');
```

```
UPDATE users  
SET name='Rahul'  
WHERE id=1;
```

```
DELETE FROM users  
WHERE id=5;
```

1.5 SQL vs PostgreSQL

SQL is the language you write.

PostgreSQL is the database software that executes SQL statements.

Think of it as:

Application → PostgreSQL → Database

Chapter Summary

After this chapter you should understand:

- What a database is
- Why databases are needed
- CRUD
- DBMS
- RDBMS
- Difference between SQL and PostgreSQL

Interview Questions

1. What is a database?
2. Why do we use databases instead of arrays?
3. What is CRUD?

4. What is a DBMS?
5. What is an RDBMS?
6. Difference between SQL and PostgreSQL?
7. Is SQL a programming language?
8. Name some DBMS software.

Practice Exercises

1. Explain CRUD in your own words.
2. Name three applications that use databases.
3. Explain SQL vs PostgreSQL without looking at the notes.
4. Why is JavaScript array storage not suitable for permanent application data?

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery
Chapter 2: SQL Basics (CRUD)

2.1 What is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases.

Almost every backend application uses SQL directly or indirectly through an ORM like Prisma.

2.2 CRUD Operations

Every backend application performs four basic operations:

- Create
- Read
- Update
- Delete

These map directly to SQL statements:

INSERT → Create

SELECT → Read

UPDATE → Update

DELETE → Delete

2.3 SELECT - Read Data

The SELECT statement retrieves data from a table.

Examples:

```
SELECT * FROM users;
```

```
SELECT name,email FROM users;
```

```
SELECT id,name,email FROM users;
```

2.4 INSERT - Create Data

INSERT adds new rows.

Example:

```
INSERT INTO users(name,email)  
VALUES ('Nikhil','nikhil@gmail.com');
```

Insert multiple rows:

```
INSERT INTO users(name,email)  
VALUES  
('Rahul','rahul@gmail.com'),  
('Aman','aman@gmail.com');
```

2.5 UPDATE - Modify Data

UPDATE changes existing rows.

UPDATE users

```
SET name='Nikhil Singh'  
WHERE id=1;
```

Always use a WHERE clause unless you intentionally want to update every row.

2.6 DELETE - Remove Data

DELETE removes rows.

```
DELETE FROM users  
WHERE id=5;
```

Without WHERE:

```
DELETE FROM users;
```

This deletes every record in the table.

2.7 TRUNCATE vs DELETE

DELETE removes rows one by one and can use WHERE.

TRUNCATE removes all rows quickly and cannot filter with WHERE.

Use TRUNCATE when you want to empty the entire table.

2.8 Best Practices

- ✓ Always use WHERE with UPDATE and DELETE unless you really intend to affect every row.
- ✓ Give columns meaningful names.
- ✓ Avoid SELECT * in production APIs if you only need a few columns.
- ✓ Use parameterized queries in applications to prevent SQL injection.

Interview Questions

1. What does CRUD stand for?
2. Difference between DELETE and TRUNCATE?
3. Why is WHERE important with UPDATE?
4. Can INSERT add multiple rows?
5. Why avoid SELECT * in some APIs?

Practice Exercises

1. Write a query to fetch all users.
2. Insert a new employee with name and salary.
3. Update an email by id.
4. Delete a user by id.
5. Explain the difference between DELETE and TRUNCATE in your own words.

Chapter Summary

After this chapter you should understand:

- SQL CRUD operations
- SELECT
- INSERT
- UPDATE
- DELETE
- TRUNCATE
- Safe usage of WHERE

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery
Chapter 3: Filtering Data

3.1 Why Filtering Matters

Most applications do not need every row from a table. Instead, we filter data to return only what we need. Examples include searching for a user by email, finding products in a price range, or showing only active users.

3.2 WHERE Clause

The WHERE clause filters rows.

Examples:

```
SELECT * FROM users WHERE id = 1;  
SELECT * FROM users WHERE age > 18;  
SELECT * FROM users WHERE salary >= 50000;
```

3.3 Comparison Operators

=
!=
>
<
>=
<=

Examples:

```
SELECT * FROM employees WHERE salary > 60000;  
SELECT * FROM users WHERE id != 5;
```

3.4 LIKE Operator

LIKE is used for pattern matching.

Starts with:

```
SELECT * FROM users WHERE name LIKE 'Rah%';
```

Ends with:

```
SELECT * FROM users WHERE email LIKE '%gmail.com';
```

Contains:

```
SELECT * FROM users WHERE email LIKE '%gmail%';
```

3.5 IN Operator

IN checks multiple values.

```
SELECT * FROM users  
WHERE id IN (1,2,5);
```

Equivalent to multiple OR conditions.

3.6 BETWEEN

BETWEEN checks a range.

```
SELECT * FROM employees  
WHERE salary BETWEEN 50000 AND 80000;
```

BETWEEN is inclusive.

3.7 DISTINCT

DISTINCT removes duplicate values.

```
SELECT DISTINCT department  
FROM employees;
```

3.8 NULL

NULL represents missing or unknown data.

```
SELECT * FROM users  
WHERE phone IS NULL;
```

```
SELECT * FROM users  
WHERE phone IS NOT NULL;
```

3.9 Combining Conditions

Use AND and OR.

```
SELECT * FROM users  
WHERE age >= 18  
AND city='Delhi';
```

```
SELECT * FROM users  
WHERE city='Delhi'  
OR city='Noida';
```

Common Mistakes

- Using = NULL instead of IS NULL.
- Forgetting quotes around strings.
- Misusing LIKE patterns.
- Forgetting WHERE in UPDATE or DELETE.

Interview Questions

1. Difference between LIKE and = ?
2. Difference between IN and multiple OR conditions?
3. Is BETWEEN inclusive?
4. Why do we use DISTINCT?
5. Difference between NULL and an empty string?

Practice Exercises

1. Find all Gmail users.
2. Find users whose names start with 'A'.
3. Find employees with salary between 40000 and 90000.
4. Find unique departments.
5. Find users whose phone number is NULL.

Chapter Summary

You learned:

- WHERE
- Comparison operators
- LIKE
- IN
- BETWEEN
- DISTINCT
- NULL
- AND / OR

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 4: Sorting & Pagination

4.1 Why Sorting and Pagination?

Real-world applications rarely display all records at once. Sorting helps present data in a meaningful order, while pagination improves performance by loading only the required rows.

4.2 ORDER BY

ORDER BY sorts query results.

Ascending (default):

```
SELECT * FROM users  
ORDER BY name ASC;
```

Descending:

```
SELECT * FROM users  
ORDER BY salary DESC;
```

4.3 Sorting by Multiple Columns

SQL allows sorting using more than one column.

Example:

```
SELECT *  
FROM employees  
ORDER BY department ASC,  
salary DESC;
```

Employees are first grouped by department, then sorted by salary within each department.

4.4 LIMIT

LIMIT restricts the number of rows returned.

Example:

```
SELECT *  
FROM users  
LIMIT 10;
```

Useful for dashboards, search results and APIs.

4.5 OFFSET

OFFSET skips rows before returning results.

Example:

```
SELECT *  
FROM users  
LIMIT 10 OFFSET 20;
```

This skips the first 20 rows and returns the next 10.

4.6 Pagination

Suppose each page displays 10 users.

Page 1:
LIMIT 10 OFFSET 0

Page 2:
LIMIT 10 OFFSET 10

Page 3:
LIMIT 10 OFFSET 20

This is the basis of pagination in most REST APIs.

4.7 Performance Considerations

Loading thousands of rows at once:

- Increases query time
- Uses more memory
- Slows the frontend

Pagination improves:

- User experience
- Server performance
- Network usage

4.8 SQL ↔ Prisma Mapping

SQL:
ORDER BY -> Prisma: orderBy
LIMIT -> Prisma: take
OFFSET -> Prisma: skip

Example:

```
await prisma.user.findMany({
  orderBy: { name: 'asc' },
  skip: 20,
  take: 10
});
```

Common Mistakes

- Forgetting ASC/DESC when readability matters.
- Using LIMIT without ORDER BY when consistent ordering is required.
- Loading all rows instead of paginating large datasets.

Interview Questions

1. Why is pagination important?
2. Difference between LIMIT and OFFSET?
3. Can ORDER BY sort multiple columns?
4. What Prisma methods correspond to LIMIT and OFFSET?
5. Why should APIs avoid returning thousands of records?

Practice Exercises

1. Return users ordered by age (youngest first).
2. Return employees ordered by salary (highest first).

3. Return only the first 5 users.
4. Return rows 21-30 using LIMIT and OFFSET.
5. Translate LIMIT/OFFSET into Prisma take/skip.

Chapter Summary

You learned:

- ORDER BY
- ASC / DESC
- Multi-column sorting
- LIMIT
- OFFSET
- Pagination
- SQL ↔ Prisma mapping

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 5: Aggregate Functions

5.1 What are Aggregate Functions?

Aggregate functions perform calculations on multiple rows and return a single result. They are commonly used for reporting, analytics and dashboards.

5.2 COUNT()

COUNT() returns the number of rows.

Examples:

```
SELECT COUNT(*) FROM users;
```

```
SELECT COUNT(email) FROM users;
```

```
SELECT COUNT(DISTINCT department) FROM employees;
```

Use COUNT(*) to count all rows regardless of NULL values.

5.3 SUM()

SUM() adds numeric values.

```
SELECT SUM(salary)
```

```
FROM employees;
```

Useful for total sales, revenue and salaries.

5.4 AVG()

AVG() calculates the average.

```
SELECT AVG(salary)
```

```
FROM employees;
```

Often used in business reports.

5.5 MIN() and MAX()

MIN() returns the smallest value.

MAX() returns the largest value.

```
SELECT MIN(salary) FROM employees;
```

```
SELECT MAX(salary) FROM employees;
```

Can also be used with dates and strings.

5.6 Combining with WHERE

Aggregate functions are often combined with WHERE.

Example:

```
SELECT COUNT(*)
```

```
FROM users
```

```
WHERE city='Delhi';
```

```
SELECT AVG(salary)
FROM employees
WHERE department='IT';
```

5.7 Aggregate Functions with GROUP BY

Aggregates become more powerful with GROUP BY.

Example:
SELECT department,
COUNT(*)
FROM employees
GROUP BY department;

Each department gets its own count.

5.8 SQL ↔ Prisma

SQL:
SELECT COUNT(*) FROM users;

Prisma:
await prisma.user.count();

SQL:
SELECT AVG(salary) FROM employees;

Prisma uses aggregate():

```
await prisma.employee.aggregate({
  _avg: {
    salary: true
  }
});
```

Common Mistakes

- Using aggregate functions without understanding GROUP BY.
- Expecting COUNT(column) to count NULL values.
- Forgetting WHERE when filtering before aggregation.

Interview Questions

1. Difference between COUNT(*) and COUNT(column)?
2. When would you use AVG()?
3. Can MIN() work on dates?
4. Why are aggregate functions important in reporting?
5. Which Prisma methods correspond to COUNT and AVG?

Practice Exercises

1. Count total users.
2. Find the highest salary.
3. Find the lowest product price.
4. Calculate the average employee salary.
5. Find the total salary of employees in the IT department.

Chapter Summary

You learned:

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()
- Aggregation with WHERE
- Introduction to GROUP BY
- SQL ↔ Prisma mapping

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 6: GROUP BY & HAVING

6.1 Why GROUP BY?

Aggregate functions calculate a single value. GROUP BY divides rows into groups so an aggregate can be calculated for each group separately.

6.2 GROUP BY Syntax

Example:

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

Result:

IT -> 10

HR -> 6

Sales -> 15

6.3 Multiple Columns

You can group by more than one column.

Example:

```
SELECT department, city, COUNT(*)  
FROM employees  
GROUP BY department, city;
```

Each unique department-city combination becomes a separate group.

6.4 HAVING

HAVING filters groups after aggregation.

Example:

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) >= 5;
```

Only departments with at least five employees are returned.

6.5 WHERE vs HAVING

WHERE filters individual rows before grouping.

HAVING filters groups after GROUP BY.

Example:

```
SELECT department, AVG(salary)  
FROM employees
```

```
WHERE salary > 30000
GROUP BY department
HAVING AVG(salary) > 50000;
```

Execution Order

SQL generally executes in this order:

```
FROM
→ WHERE
→ GROUP BY
→ HAVING
→ SELECT
→ ORDER BY
→ LIMIT
```

6.6 SQL ↔ Prisma

Prisma supports grouping with `groupBy()`.

Example:

```
await prisma.employee.groupBy({
  by: ['department'],
  _count: {
    department: true
  }
});
```

Common Mistakes

- Using WHERE with aggregate functions instead of HAVING.
- Selecting columns that are neither grouped nor aggregated.
- Forgetting GROUP BY when using aggregates by category.

Interview Questions

1. Difference between WHERE and HAVING?
2. Why do we use GROUP BY?
3. Can GROUP BY use multiple columns?
4. In what order are WHERE and HAVING executed?
5. Which Prisma method performs grouping?

Practice Exercises

1. Count employees in each department.
2. Show departments with more than 10 employees.
3. Find the average salary for each department.
4. Display only departments whose average salary exceeds 60000.
5. Translate a GROUP BY query into Prisma `groupBy()`.

Chapter Summary

You learned:

- GROUP BY
- HAVING
- WHERE vs HAVING
- Multi-column grouping
- SQL execution order
- Prisma `groupBy()`

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 7: SQL Joins

7.1 Why Joins?

In real-world databases, related data is stored in different tables to avoid duplication. Joins allow us to combine those tables into one result set.

Example Tables

Users

id	name
1	Nikhil
2	Rahul

Orders

id	product	user_id
1	Laptop	1
2	Mouse	1
3	Keyboard	2

7.2 INNER JOIN

Returns only matching rows.

```
SELECT users.name, orders.product
FROM users
INNER JOIN orders
ON users.id = orders.user_id;
```

Result:

Nikhil -> Laptop

Nikhil -> Mouse

Rahul -> Keyboard

7.3 LEFT JOIN

Returns all rows from the left table and matching rows from the right table.

If no match exists, NULL values are returned for the right table.

7.4 RIGHT JOIN

Returns all rows from the right table and matching rows from the left table.

Less commonly used because LEFT JOIN is usually easier to read.

7.5 FULL OUTER JOIN

Returns every row from both tables.

Matching rows are combined.

Non-matching rows contain NULL values.

7.6 SELF JOIN

A table joins with itself.

Useful for:

- Employees and managers
- Categories and parent categories
- Referral systems

7.7 CROSS JOIN

Produces every possible combination of rows.

Example:

3 colors × 2 sizes = 6 combinations.

Use carefully because result sets can become very large.

7.8 Foreign Keys

Joins rely on relationships.

Users.id (Primary Key)

↓

Orders.user_id (Foreign Key)

The foreign key links both tables together.

7.9 SQL ↔ Prisma

SQL:

```
SELECT users.name, orders.product
FROM users
JOIN orders
ON users.id = orders.user_id;
```

Prisma:

```
await prisma.user.findMany({
  include: {
    orders: true
  }
});
```

To return selected fields only:

```
await prisma.user.findMany({
  select: {
    name: true,
    orders: {
      select: {
        product: true
      }
    }
  }
});
```

When to Use Each Join

INNER JOIN -> Only matching records.
LEFT JOIN -> Keep all left table records.
RIGHT JOIN -> Keep all right table records.
FULL JOIN -> Keep everything.
SELF JOIN -> Same table relationships.
CROSS JOIN -> Every possible combination.

Common Mistakes

- Forgetting the ON condition.
- Joining on the wrong columns.
- Using INNER JOIN when LEFT JOIN is required.
- Returning unnecessary columns with SELECT *.

Interview Questions

1. Difference between INNER and LEFT JOIN?
2. What is a Foreign Key?
3. Why do we normalize data into multiple tables?
4. What is a SELF JOIN?
5. How do joins relate to Prisma include?

Practice Exercises

1. List every user with their orders.
2. Show all users, even if they have no orders.
3. Find employees and their managers using SELF JOIN.
4. Write a JOIN between products and categories.
5. Convert an INNER JOIN query into Prisma include().

Chapter Summary

You learned:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- SELF JOIN
- CROSS JOIN
- Foreign Keys
- SQL ↔ Prisma relation mapping

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 8: Indexes & Query Optimization

8.1 Why Do We Need Indexes?

Imagine searching for one student's record in a notebook containing one million names.

Without an index, every row may need to be checked. An index works like the index at the back of a book, allowing the database to find matching rows much faster.

8.2 What is an Index?

An index is a special data structure maintained by the database to speed up searching and sorting on one or more columns.

Example:

```
CREATE INDEX idx_users_email  
ON users(email);
```

8.3 How Indexes Help

Queries such as:

```
SELECT * FROM users  
WHERE email='nikhil@gmail.com';
```

become much faster when an index exists on the email column.

8.4 Drawbacks of Indexes

Indexes are not free.

- They consume storage.
- INSERT, UPDATE and DELETE become slightly slower because indexes must also be updated.

Create indexes only when they provide real value.

8.5 When Should You Create an Index?

Good candidates:

- Primary Keys
- Foreign Keys
- Frequently searched columns
- Columns used in ORDER BY
- Columns used in JOIN conditions

8.6 When Should You Avoid Indexes?

Avoid indexing:

- Very small tables
- Columns that change frequently
- Columns with very few unique values unless testing shows a benefit

8.7 Unique & Composite Indexes

Unique Index:

```
CREATE UNIQUE INDEX idx_email  
ON users(email);
```

Composite Index:
`CREATE INDEX idx_city_department
ON employees(city, department);`

Composite indexes help queries that filter on both columns.

8.8 Query Optimization Tips

- Select only required columns instead of `SELECT *`.
- Filter rows with `WHERE`.
- Use `LIMIT` for pagination.
- Avoid unnecessary joins.
- Index frequently searched columns.
- Analyze slow queries before optimizing.

8.9 EXPLAIN (Introduction)

PostgreSQL provides `EXPLAIN` to show how a query will be executed.

Example:

```
EXPLAIN  
SELECT * FROM users  
WHERE email='nikhil@gmail.com';
```

It helps identify whether PostgreSQL uses an index or performs a full table scan.

8.10 SQL ↔ Prisma

Prisma automatically benefits from indexes created in the database.
No special Prisma query is required.

Example:

```
await prisma.user.findUnique({  
  where: {  
    email: 'nikhil@gmail.com'  
  }  
});
```

If email is indexed, PostgreSQL uses that index automatically.

Common Mistakes

- Adding indexes to every column.
- Forgetting that indexes slow writes.
- Using `SELECT *` unnecessarily.
- Optimizing without measuring performance.

Interview Questions

1. What is an index?
2. Why are indexes useful?
3. What are the disadvantages of indexes?
4. What is a composite index?
5. Does Prisma create indexes automatically for every field?

Practice Exercises

1. Create an index on the email column.
2. Create a composite index on city and department.
3. Identify three columns in a social media app that should be indexed.
4. Explain why primary keys are indexed.
5. Use EXPLAIN on a simple SELECT query.

Chapter Summary

You learned:

- What indexes are
- Why indexes improve performance
- Unique indexes
- Composite indexes
- Query optimization basics
- EXPLAIN
- How Prisma benefits from database indexes

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 9: Database Design & Relationships

9.1 Why Database Design Matters

A good database design reduces duplicate data, improves consistency, and makes applications easier to maintain. Before writing SQL, think about how data is related.

9.2 Tables and Entities

Each table usually represents one real-world entity.

Examples:

- Users
- Products
- Orders
- Categories
- Messages

Each row represents one record, and each column stores one property.

9.3 Primary Key

A Primary Key uniquely identifies every row.

Example:

```
id INT PRIMARY KEY
```

Best Practice:

Use an auto-incrementing integer or UUID depending on your application's requirements.

9.4 Foreign Key

A Foreign Key connects one table to another.

Example:

```
Orders.user_id  
→ Users.id
```

Foreign keys maintain referential integrity and prevent invalid relationships.

9.5 Relationship Types

One-to-One

Example: User → Passport

One-to-Many

Example: User → Orders

Many-to-Many

Example: Students ↔ Courses

This is implemented using a junction table (StudentCourses).

9.6 Normalization

Normalization organizes data to reduce duplication.

Instead of storing a user's address in every order, store it once and reference the user.

Benefits:

- Less duplicate data
- Easier updates
- Better consistency

9.7 Example Schema

Users

(id, name, email)

Orders

(id, product, user_id)

Relationship:

Users.id → Orders.user_id

A single user can have many orders.

9.8 SQL ↔ Prisma

SQL:

Orders.user_id REFERENCES Users(id)

Prisma:

```
model User {  
  id Int @id @default(autoincrement())  
  orders Order[]  
}
```

```
model Order {  
  id Int @id @default(autoincrement())  
  userId Int  
  user User @relation(fields:[userId], references:[id])  
}
```

Best Practices

- Give tables meaningful names.
- Use Primary Keys.
- Use Foreign Keys for relationships.
- Avoid duplicate data.
- Index frequently searched foreign keys.
- Keep one responsibility per table.

Common Mistakes

- Storing the same information in multiple tables.
- Missing foreign key constraints.
- Using one huge table for unrelated data.
- Ignoring relationships during design.

Interview Questions

1. What is database normalization?
2. Difference between Primary Key and Foreign Key?

3. Explain one-to-many with an example.
4. Why are foreign keys important?
5. How are relationships represented in Prisma?

Practice Exercises

1. Design tables for a library system.
2. Create a one-to-many relationship between customers and orders.
3. Draw a many-to-many relationship between students and courses.
4. Convert a SQL relationship into Prisma models.
5. Identify duplicate data in a sample schema and normalize it.

Chapter Summary

You learned:

- Database design
- Primary & Foreign Keys
- One-to-One
- One-to-Many
- Many-to-Many
- Normalization
- SQL ↔ Prisma relationships

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 1: SQL Mastery

Chapter 10: SQL Interview Guide, Cheat Sheet & Mini Project

10.1 SQL Revision Cheat Sheet

CRUD:

SELECT, INSERT, UPDATE, DELETE

Filtering:

WHERE, LIKE, IN, BETWEEN, IS NULL

Sorting:

ORDER BY ASC/DESC

Pagination:

LIMIT, OFFSET

Aggregation:

COUNT, SUM, AVG, MIN, MAX

Grouping:

GROUP BY, HAVING

Joins:

INNER, LEFT, RIGHT, FULL, SELF, CROSS

Optimization:

Indexes, EXPLAIN

10.2 Frequently Asked Interview Questions

1. What is SQL?
2. Difference between SQL and PostgreSQL?
3. What is a Primary Key?
4. What is a Foreign Key?
5. Difference between WHERE and HAVING?
6. Difference between DELETE and TRUNCATE?
7. What is an Index?
8. Difference between INNER JOIN and LEFT JOIN?
9. What is Normalization?
10. Why do we use LIMIT and OFFSET?

10.3 Mini Project - Employee Management System

Tables:

Employees

Departments

Features:

- Add Employee
- Update Employee
- Delete Employee
- Search by name
- Filter by department
- Sort by salary
- Paginate employee list

Useful SQL:

```
SELECT * FROM employees WHERE name LIKE 'Nik%';
```

```
SELECT * FROM employees ORDER BY salary DESC;
```

```
SELECT * FROM employees LIMIT 10 OFFSET 0;
```

10.4 Best Practices

- Use parameterized queries in applications.
- Avoid SELECT * unless required.
- Always use WHERE with UPDATE and DELETE.
- Create indexes on frequently searched columns.
- Normalize your schema.
- Keep SQL readable and consistent.

10.5 Common Beginner Mistakes

- Forgetting WHERE in UPDATE/DELETE.
- Confusing WHERE and HAVING.
- Forgetting JOIN conditions.
- Returning unnecessary columns.
- Ignoring indexes on large tables.

10.6 Final Self-Assessment

If you can confidently explain and write:

- ✓ CRUD queries
- ✓ Filtering
- ✓ Sorting
- ✓ Pagination
- ✓ Aggregate Functions
- ✓ GROUP BY & HAVING
- ✓ JOINS
- ✓ Indexes
- ✓ Database Relationships

then you have a solid SQL foundation for backend development.

Volume 1 Completed!

Congratulations!

You have completed Volume 1: SQL Mastery.

Next Volume:

PostgreSQL & node-postgres

Topics include:

- PostgreSQL installation
- psql
- pgAdmin
- Pool vs Client
- Parameterized queries
- Express integration
- Seed scripts
- SQL Injection prevention